Numbers Representation in Computer Science

Sorbonne Université – Master SESI – UM4IN111 – PPSE

Adrien Cassagne

January 20, 2026

Table of Contents

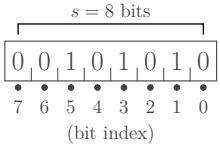
- Integers
- Floating-point – IEEE 754
- AI Floating-point Formats for DNNs
- Fixed-point

Natural Integers – Decimal to Base 2 Numbers

Convert 42 to binary, from the right to the left:

1. If the number is **even put a 0** else if it is **odd put a 1**
2. **Divide** the current number **by 2** and **redo the first step** on the result

— 42 is even $\rightarrow b_0 = 0,\ \lfloor 42/2 \rfloor = 21$
— 21 is odd $\rightarrow b_1 = 1,\ \lfloor 21/2 \rfloor = 10$
— 10 is even $\rightarrow b_2 = 0,\ \lfloor 10/2 \rfloor = 5$
— 5 is odd $\rightarrow b_3 = 1,\ \lfloor 5/2 \rfloor = 2$
— 2 is even $\rightarrow b_4 = 0,\ \lfloor 2/2 \rfloor = 1$
— 1 is odd $\rightarrow b_5 = 1,\ \lfloor 1/2 \rfloor = 0 \Rightarrow$ STOP!

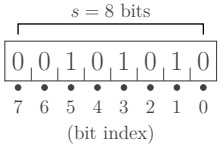


Natural Integers – Decimal to Base 2 Numbers

Convert 42 to binary, from the right to the left:

1. If the number is **even put a 0** else if it is **odd put a 1**
2. **Divide** the current number **by 2** and **redo the first step** on the result

— 42 is even $\rightarrow b_0 = 0,\ \lfloor 42/2 \rfloor = 21$
— 21 is odd $\rightarrow b_1 = 1,\ \lfloor 21/2 \rfloor = 10$
— 10 is even $\rightarrow b_2 = 0,\ \lfloor 10/2 \rfloor = 5$
— 5 is odd $\rightarrow b_3 = 1,\ \lfloor 5/2 \rfloor = 2$
— 2 is even $\rightarrow b_4 = 0,\ \lfloor 2/2 \rfloor = 1$
— 1 is odd $\rightarrow b_5 = 1,\ \lfloor 1/2 \rfloor = 0 \Rightarrow$ STOP!



Convert back the **binary to decimal**:

- $0 \times 2^7 + 0 \times 2^6 + \mathbf{1} \times 2^5 + 0 \times 2^4 + \mathbf{1} \times 2^3 + 0 \times 2^2 + \mathbf{1} \times 2^1 + 0 \times 2^0 = 32 + 8 + 2 = 42$
- In general: $\mathcal{N} = \sum_{i=0}^{s-1} b_i \times 2^i$, with s the capacity in number of bits ($s = 8$ here)

Natural Integers – Warmup Tasks

- Code in base 2 the following numbers (capacity $s = 8$): 112, 43
- What are the following numbers in base 10?

$s = 8$ bits

0	1	0	0	1	0	1	0
7	6	5	4	3	2	1	0

(bit index)

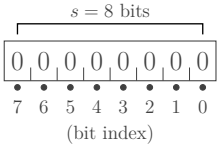
$s = 8$ bits

1	1	0	0	0	0	0	1
7	6	5	4	3	2	1	0

(bit index)

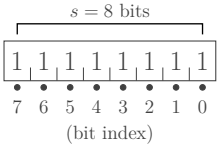
Natural Integers – Ranges

Smallest number is 0 (when $s = 8$)



$$0 \times 2^7 + 0 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 0$$

Biggest number is 255 (when $s = 8$)



$$1 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 255$$

In the general case, natural integers **interval of represented values** is: $[0; 2^s - 1]$.

- In CS, the natural integers are generally called **unsigned integers**

Natural Integers – Overflow

Overflow happens when the addition (or the subtraction) goes **out of the represented values interval**

- Examples on 8-bit unsigned integers ($s = 8$)

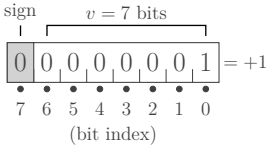
$$\boxed{1\,1\,1\,1\,1\,1\,1\,1} \; +1 = \boxed{0\,0\,0\,0\,0\,0\,0\,0} \quad (255 + 1 = 0)$$

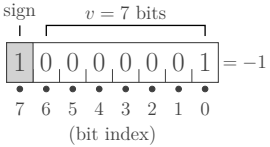
$$\boxed{0\,0\,0\,0\,0\,0\,0\,0} \; -1 = \boxed{1\,1\,1\,1\,1\,1\,1\,1} \quad (0 - 1 = 255)$$

Relative Integers – Symmetric with Sign Bit

- Integers that **can be negative**
- Also called **signed integers** or just **integers** in computer science

Idea: **Add a sign bit**

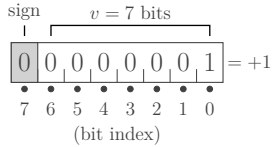


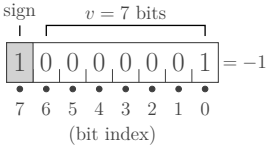


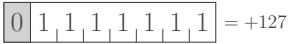
Relative Integers – Symmetric with Sign Bit

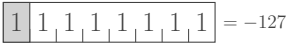
- Integers that **can be negative**
- Also called **signed integers** or just **integers** in computer science

Idea: **Add a sign bit**







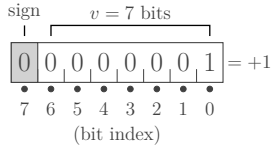


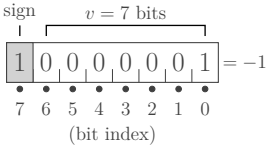
- Interval is $[-127; 127]$ or $[-(2^{s-1} - 1); 2^{s-1} - 1]$

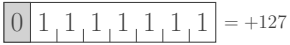
Relative Integers – Symmetric with Sign Bit

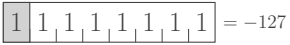
- Integers that **can be negative**
- Also called **signed integers** or just **integers** in computer science

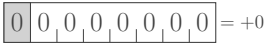
Idea: **Add a sign bit**

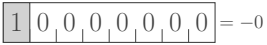










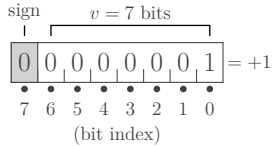


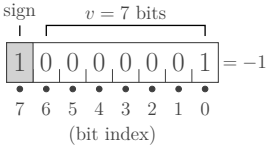
- Interval is $[-127; 127]$ or $[-(2^{s-1} - 1); 2^{s-1} - 1]$
- Zero has two representations ($+0$ and -0)

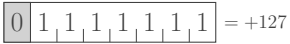
Relative Integers – Symmetric with Sign Bit

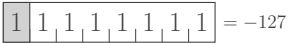
- Integers that **can be negative**
- Also called **signed integers** or just **integers** in computer science

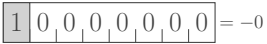
Idea: **Add a sign bit**









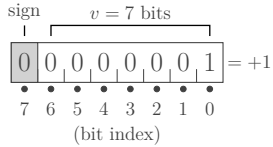


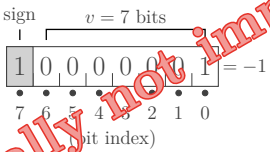
- Interval is $[-127; 127]$ or $[-(2^{s-1} - 1); 2^{s-1} - 1]$
- Zero has two representations ($+0$ and -0)
- **Problem**: The HW additionner cannot be easily reused for the subtraction

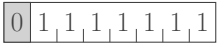
Relative Integers – Symmetric with Sign Bit

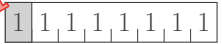
- Integers that **can be negative**
- Also called **signed integers** or just **integers** in computer science

Idea: **Add a sign bit**









- Interval is $[-127; 127]$ or $[-(2^{s-1} - 1); 2^{s-1} - 1]$
- Zero has two representations ($+0$ and -0)
- **Problem**: The HW additionner cannot be easily reused for the subtraction

This is generally not implemented in HW!

Relative Integers – Two's Complement

We would like to have

$$
\begin{array}{rl}
 & \boxed{0}\boxed{0\ 0\ 0\ 0\ 0\ 0\ 0\ 1} = +1 \\
+ & \boxed{1}\boxed{?\ ?\ ?\ ?\ ?\ ?\ ?\ ?} = -1 \\
\hline
 & \boxed{0}\boxed{0\ 0\ 0\ 0\ 0\ 0\ 0\ 0} = +0
\end{array}
$$

Relative Integers – Two's Complement

We would like to have

$$
\begin{array}{rl}
 & \boxed{0}\boxed{0\,0\,0\,0\,0\,0\,0\,1} = +1 \\
+ & \boxed{1}\boxed{?\,?\,?\,?\,?\,?\,?\,?} = -1 \\
\hline
 & \boxed{0}\boxed{0\,0\,0\,0\,0\,0\,0\,0} = +0
\end{array}
$$

Solution

$$\boxed{1}\boxed{1\,1\,1\,1\,1\,1\,1\,1} = -1$$

Relative Integers – Two's Complement

We would like to have

$$
\begin{array}{rl}
 & \boxed{0}\boxed{0\,0\,0\,0\,0\,0\,0\,1} = +1 \\
+ & \boxed{1}\boxed{?\,?\,?\,?\,?\,?\,?\,?} = -1 \\
\hline
 & \boxed{0}\boxed{0\,0\,0\,0\,0\,0\,0\,0} = +0
\end{array}
$$

$$+1 = \boxed{0}\boxed{0\,0\,0\,0\,0\,0\,0\,1}$$

Solution

$$\boxed{1}\boxed{1\,1\,1\,1\,1\,1\,1\,1} = -1$$

Two's complement

1. Start from positive number

Relative Integers – Two's Complement

We would like to have

$$
\begin{array}{rl}
 & \boxed{0}\boxed{0\,0\,0\,0\,0\,0\,0\,1} = +1 \\
+ & \boxed{1}\boxed{?\,?\,?\,?\,?\,?\,?\,?} = -1 \\
\hline
 & \boxed{0}\boxed{0\,0\,0\,0\,0\,0\,0\,0} = +0
\end{array}
$$

Solution

$$\boxed{1}\boxed{1\,1\,1\,1\,1\,1\,1\,1} = -1$$

Two's complement

1. Start from positive number
2. Calculate 1's complement

$$+1 = \boxed{0}\boxed{0\,0\,0\,0\,0\,0\,0\,1} \qquad \overline{+1} = \boxed{1}\boxed{1\,1\,1\,1\,1\,1\,1\,0}$$

Relative Integers – Two's Complement

We would like to have

$$
\begin{array}{rl}
 & \boxed{0}\boxed{0\,0\,0\,0\,0\,0\,0\,1} = +1 \\
+ & \boxed{1}\boxed{?\,?\,?\,?\,?\,?\,?\,?} = -1 \\
\hline
 & \boxed{0}\boxed{0\,0\,0\,0\,0\,0\,0\,0} = +0
\end{array}
$$

Solution

$$\boxed{1}\boxed{1\,1\,1\,1\,1\,1\,1\,1} = -1$$

Two's complement

1. Start from positive number
2. Calculate 1's complement
3. Add one

$$+1 = \boxed{0}\boxed{0\,0\,0\,0\,0\,0\,0\,1}$$

$$\overline{+1} = \boxed{1}\boxed{1\,1\,1\,1\,1\,1\,1\,0}$$

$$\overline{+1} + 1 = \boxed{1}\boxed{1\,1\,1\,1\,1\,1\,1\,1} = -1$$

Relative Integers – Two's Complement

We would like to have

$$
\begin{array}{rl}
 & \boxed{0}\boxed{0\ 0\ 0\ 0\ 0\ 0\ 0\ 1} = +1 \\
+ & \boxed{1}\boxed{?\ ?\ ?\ ?\ ?\ ?\ ?\ ?} = -1 \\
\hline
 & \boxed{0}\boxed{0\ 0\ 0\ 0\ 0\ 0\ 0\ 0} = +0
\end{array}
$$

Solution

$$\boxed{1}\boxed{1\ 1\ 1\ 1\ 1\ 1\ 1\ 1} = -1$$

Two's complement

1. Start from positive number
2. Calculate 1's complement
3. Add one

$$+1 = \boxed{0}\boxed{0\ 0\ 0\ 0\ 0\ 0\ 0\ 1} \qquad \overline{+1} = \boxed{1}\boxed{1\ 1\ 1\ 1\ 1\ 1\ 1\ 0} \qquad \overline{+1} + 1 = \boxed{1}\boxed{1\ 1\ 1\ 1\ 1\ 1\ 1\ 1} = -1$$

From **binary to decimal**:

- $\mathcal{N} = b_7 \times -2^7 + \sum_{i=0}^{6} b_i \times 2^i$ or $\mathcal{N} = b_{s-1} \times -2^{s-1} + \sum_{i=0}^{s-2} b_i \times 2^i$
- $\mathcal{N}_{-1} = -2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = -128 + 64 + 32 + 16 + 8 + 4 + 2 + 1$

Relative Integers – Two's Complement

We would like to have

$$
\begin{array}{rl}
 & \boxed{0}\boxed{0\,0\,0\,0\,0\,0\,0\,1} = +1 \\
+ & \boxed{1}\boxed{?\,?\,?\,?\,?\,?\,?\,?} = -1 \\
\hline
 & \boxed{0}\boxed{0\,0\,0\,0\,0\,0\,0\,0} = +0
\end{array}
$$

Solution

$$\boxed{1}\boxed{1\,1\,1\,1\,1\,1\,1\,1} = -1$$

Two's complement

1. Start from positive number
2. Calculate 1's complement
3. Add one

$$+1 = \boxed{0}\boxed{0\,0\,0\,0\,0\,0\,0\,1} \qquad \overline{+1} = \boxed{1}\boxed{1\,1\,1\,1\,1\,1\,1\,0} \qquad \overline{+1} + 1 = \boxed{1}\boxed{1\,1\,1\,1\,1\,1\,1\,1} = -1$$

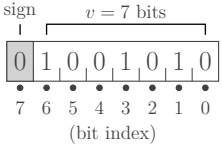
From **binary to decimal**:

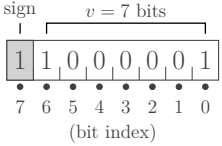
- $\mathcal{N} = b_s \times -2^s + \sum_{i=0}^{6} b_i \times 2^i$ or $\mathcal{N} = b_{s-1} \times -2^{s-1} + \sum_{i=0}^{s-2} b_i \times 2^i$
- $\mathcal{N} = -2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = -128 + 64 + 32 + 16 + 8 + 4 + 2 + 1$

Two's complement is implemented in HW!

Relative Integers – 2's Compl. Warmup Tasks

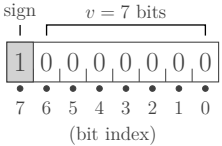
- Code in base 2 the following numbers (capacity $s = v + 1 = 8$): -112, -43
- What are the following numbers in base 10?





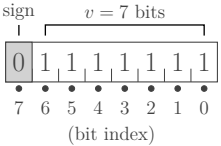
Relative Integers – 2's Complement Ranges

Smallest number is -128 (when $s = 8$)



$$-1 \times 2^7 + 0 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = -128$$

Biggest number is 127 (when $s = 8$)



$$-0 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 127$$

- In the general case, interval of represented values is: $[-2^{s-1}; 2^{s-1} - 1]$ (considering $s = v + 1$)
- **Two's complement is the standard** way to store signed integers **in today computer architectures**

Relative Integers – Overflow Problem

Overflow happens when the addition (or the subtraction) goes **out of the represented values interval**.

- Examples on 8-bit signed integers ($s = 8$) with $[-128; 127]$ interval

$$\text{(-128) + (-1)} = \boxed{1\,0\,0\,0\,0\,0\,0\,0} + \boxed{1\,1\,1\,1\,1\,1\,1\,1} = \boxed{0\,1\,1\,1\,1\,1\,1\,1} = 127$$

$$127 + 1 = \boxed{0\,1\,1\,1\,1\,1\,1\,1} + \boxed{0\,0\,0\,0\,0\,0\,0\,1} = \boxed{1\,0\,0\,0\,0\,0\,0\,0} = -128$$

Saturated Arithmetic

- Specific instructions that **avoid overflowing**
- Generally available in **SIMD ISAs** and **on small capacities** (typically $s = 8$, $s = 16$ or $s = 32$)
- Common **SIMD saturated addition instructions** (C intrinsics)
  - x86[_64] – SSE, AVX & AVX-512: `_mm*_adds_[epi/epu][8/16]`
  - ARMv[7/8/9] – Neon: `vqaddq_[s/u][8/16/32]`
  - ARMv[8/9] – SVE[1/2]: `svqadd_[s/u][8/16/32]`
- On signed 8-bit integers:

$$(\text{-}128) + (\text{-}1) = \boxed{1\,0\,0\,0\,0\,0\,0\,0} + \boxed{1\,1\,1\,1\,1\,1\,1\,1} = \boxed{1\,0\,0\,0\,0\,0\,0\,0} = -128$$

$$127 + 1 = \boxed{0\,1\,1\,1\,1\,1\,1\,1} + \boxed{0\,0\,0\,0\,0\,0\,0\,1} = \boxed{0\,1\,1\,1\,1\,1\,1\,1} = 127$$

Integers – Data Types (before C99)

Signed integers

- 8-bit: `char`, `signed char`
- 16-bit: `short`, `signed int`, `signed short`, `signed short int`
- 32-bit: `int`, `signed`, `signed int`, `long`, `long int`, `signed long`, `signed long int`
- 64-bit: `long long`, `long long int`, `signed long long`, `signed long long int`

Unsigned integers

- 8-bit: `unsigned char`
- 16-bit: `unsigned short`
- 32-bit: `unsigned`, `unsigned int`
- 64-bit: `unsigned long long`, `unsigned long long int`

Problem: The data type names **do NOT contain the data size** & **A lot of data types** are in reality the same for the HW

Integers – Data Types (starting from C99)

Introduction of the new `stdint.h` header file defining the following data types

Signed integers

- 8-bit: `int8_t`
- 16-bit: `int16_t`
- 32-bit: `int32_t`
- 64-bit: `int64_t`

Unsigned integers

- 8-bit: `uint8_t`
- 16-bit: `uint16_t`
- 32-bit: `uint32_t`
- 64-bit: `uint64_t`

- Depending on the code, it might be a good idea to use them!
  - Especially when the data size really matter in the algorithm implementation

Integers – Promotion

- Promotion is something tricky and complicated in C, be humble ;-)
- The main rule you should know is that, in an expression, the little 8-bit & 16-bit integers (signed or unsigned) are converted to 32-bit signed integers by the compiler

```c
unsigned char x = 0;
unsigned char y = 1;
printf("%u\n", x - y); // prints 4294967295
                       // and not 255
```

Integers – Promotion

- Promotion is something tricky and complicated in C, be humble ;-)
- The main rule you should know is that, in an expression, the little 8-bit & 16-bit integers (signed or unsigned) are converted to 32-bit signed integers by the compiler

```c
unsigned char x = 0;
unsigned char y = 1;
printf("%u\n", x - y); // prints 4294967295
                       // and not 255
```

```c
unsigned char x = 0;
unsigned char y = 1;
printf("%u\n", (int)x - (int)y);
// prints (unsigned)((int)x - (int)y)
```

Integers – Promotion

- Promotion is something tricky and complicated in C, be humble ;-)
- The main rule you should know is that, in an expression, the little 8-bit & 16-bit integers (signed or unsigned) are converted to 32-bit signed integers by the compiler

```c
unsigned char x = 0;
unsigned char y = 1;
printf("%u\n", x - y); // prints 4294967295
                       // and not 255
```

```c
unsigned char x = 0;
unsigned char y = 1;
printf("%u\n", (int)x - (int)y);
// prints (unsigned)((int)x - (int)y)
```

```c
unsigned char x = 0;
unsigned char y = 1;
unsigned char z = x - y;
printf("%u\n", z); // prints 255
// unsigned char z = (int)x - (int)y;
// unsigned char z = (int)-1;
// unsigned char z = (unsigned char)((int)-1);
// unsigned char z = 255
```

Integers – Conversion


```c
int main(){
  unsigned int a = 1;
  signed int b = -2;
  if (a + b > 0)
    printf("OMG -1 is bigger than 0! %d\n", a + b);
  // prints "OMG -1 is bigger than 0! 4294967295"
}
```

Integers – Conversion


```c
int main(){
  unsigned int a = 1;
  signed int b = -2;
  if (a + b > 0)
    printf("OMG -1 is bigger than 0! %u\n", a + b);
  // prints "OMG -1 is bigger than 0! 4294967295"
  // a + b = (unsigned int)a + (unsigned int)b
  // a + b = 1 + 4294967294 = 4294967295
}
```

- No promotion there but the left variable (`a`) is dominating in the type interpretation: (`b`) is converted into a 32-bit unsigned integer

Integers – Conversion


```c
int main(){
  unsigned int a = 1;
  signed int b = -2;
  if (a + b > 0)
    printf("OMG -1 is bigger than 0! %u\n", a + b);
  // prints "OMG -1 is bigger than 0! 4294967295"
  // a + b = (unsigned int)a + (unsigned int)b
  // a + b = 1 + 4294967294 = 4294967295
}
```

- No promotion there but the left variable (`a`) is dominating in the type interpretation: (`b`) is converted into a 32-bit unsigned integer

- Possible solution:

```c
int main(){
  unsigned int a = 1;
  signed int b = -2;
  signed int c = a + b;
  if (c < 0)
    printf("-1 is smaller than 0! %d\n", c);
  // prints "-1 is smaller than 0! -1"
}
```

Integers – Extension

TODO

Table of Contents

2 Floating-point – IEEE 754

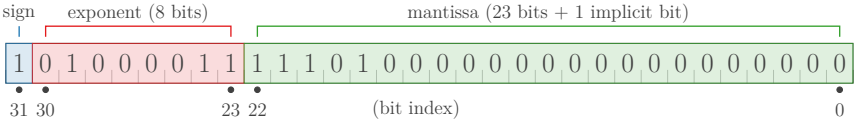
- Integers
- Floating-point – IEEE 754
- AI Floating-point Formats for DNNs
- Fixed-point

IEEE 754 – Presentation

IEEE Standard for Floating-Point Arithmetic (IEEE 754) from 1985

- Most of hardware floating-point units use the IEEE 754 standard
- Defines
  - **Arithmetic formats: Sets of binary and decimal floating-point data, which consist of finite numbers (including signed zeros and subnormal numbers), infinities, and special "not a number" values (NaNs)**
  - Interchange formats: Encodings (bit strings) that may be used to exchange floating-point data in an efficient and compact form
  - Rounding rules: Properties to be satisfied when rounding numbers during arithmetic and conversions
  - Operations: Arithmetic and other operations (such as trigonometric functions) on arithmetic formats
  - Exception handling: Indications of exceptional conditions (such as division by zero, overflow, etc.)

32-bit Floating-point – Normalized Numbers

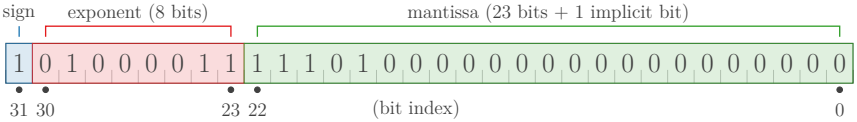


- Let s be the coefficient representing if the overall number is **positive or negative**

$$s = \begin{cases} +1, & \text{if } b_{31} = 0 \\ -1, & \text{otherwise} \end{cases}$$

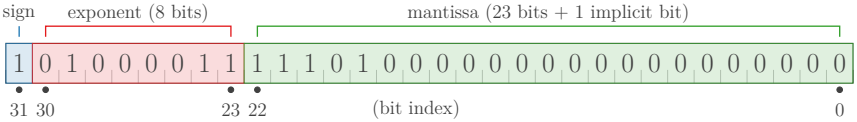
, see the sign bit b_{31} in blue

32-bit Floating-point – Normalized Numbers



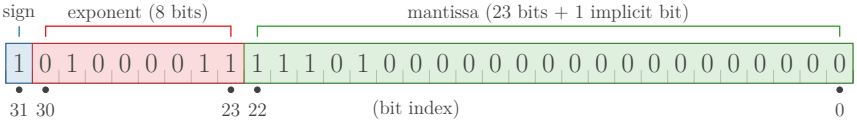
- Let s be the coefficient representing if the overall number is **positive or negative**
  $s = \begin{cases} +1, & \text{if } b_{31} = 0 \\ -1, & \text{otherwise} \end{cases}$, see the sign bit b_{31} in blue
- Let e be the **exponent of the power of two** represented by the bits $b_{[23;30]}$ in red: 2^{e-127}

32-bit Floating-point – Normalized Numbers



- Let s be the coefficient representing if the overall number is **positive or negative**
  $s = \begin{cases} +1, & \text{if } b_{31} = 0 \\ -1, & \text{otherwise} \end{cases}$, see the sign bit b_{31} in blue
- Let e be the **exponent of the power of two** represented by the bits $b_{[23;30]}$ in red: 2^{e-127}
- Let m bet the number represented by the **mantissa** (or **fraction part**) and by the bits $b_{[0;22]}$ in green: $m = 1 + \sum_{i=1}^{23} b_{23-i} \times 2^{-i}$

32-bit Floating-point – Normalized Numbers

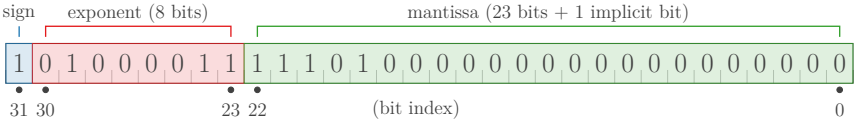


- Let s be the coefficient representing if the overall number is **positive or negative**
  $s = \begin{cases} +1, & \text{if } b_{31} = 0 \\ -1, & \text{otherwise} \end{cases}$, see the sign bit b_{31} in blue
- Let e be the **exponent of the power of two** represented by the bits $b_{[23;30]}$ in red: 2^{e-127}
- Let m bet the number represented by the **mantissa** (or **fraction part**) and by the bits $b_{[0;22]}$ in green: $m = 1 + \sum_{i=1}^{23} b_{23-i} \times 2^{-i}$
- The **general form** of the 32-bit IEEE 754 normalized numbers is

$$\mathcal{N} = s \times 2^{e-127} \times m.$$

32-bit Floating-point – Exponent

Representation of **normalized numbers**:



Let e be the value represented by the exponent in base 10:

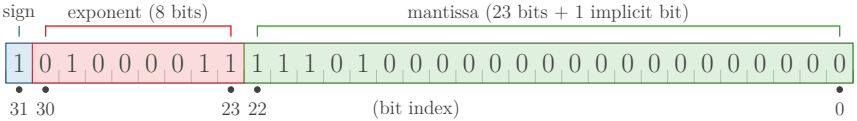
$$e = 2^6_{b_{30}} + 2^1_{b_{24}} + 2^0_{b_{23}} = 64 + 2 + 1 = 67.$$

A **biased-exponent** is used to be able **to represent negative exponents**, let n_e be the number of bits of the exponent ($n_e = 8$ here):

- This bias is $\mathcal{B} = 2^{n_e - 1} - 1 = 2^{8-1} - 1 = 2^7 - 1 = 127$
- Power of 2 value with exponent is: $2^{e-\mathcal{B}} = 2^{67-127} = 2^{-60}$
- Exponent value can vary from 1 to $2^{n_e-1} - 1$ (0 and 2^{n_e-1} are left for special values)
- Numbers of exponent between 2^{-126} and 2^{127} can be represented

32-bit Floating-point – Mantissa

Representation of **normalized numbers**:



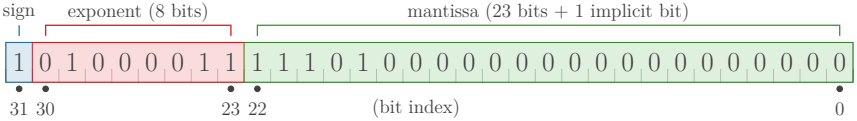
Let m be the value represented by the mantissa in base 10:

$$m = 2^0 + 2^{-1}_{b_{22}} + 2^{-2}_{b_{21}} + 2^{-3}_{b_{20}} + 2^{-5}_{b_{18}} = 1 + 0.5 + 0.25 + 0.125 + 0.03125 = 1.90625.$$

- The **implicit bit** b_{implicit} is always set to one, so that $b_{\text{implicit}} \times 2^0 = 1$ to represent numbers in the following normalized form: 1.xxx
- Bits of the mantissa represent **negative powers of two** as following: $b_{22} \times 2^{-1}, b_{21} \times 2^{-2}, ..., b_0 \times 2^{-23}$, in other words $m = 1 \times 2^0 + \sum_{i=1}^{23} b_{23-i} \times 2^{-i}$.
- Determine the maximum **precision of the floating point numbers**

32-bit Floating-point – Example Overview

Representation of **normalized numbers**:



Let e be the value represented by the exponent in base 10:

$$e = 2^6_{b_{30}} + 2^1_{b_{24}} + 2^0_{b_{23}} = 64 + 2 + 1 = 67.$$

Let m be the value represented by the mantissa in base 10:

$$m = 2^0_{b_{\text{implicit}}} + 2^{-1}_{b_{22}} + 2^{-2}_{b_{21}} + 2^{-3}_{b_{20}} + 2^{-5}_{b_{18}} = 1 + 0.5 + 0.25 + 0.125 + 0.03125 = 1.90625.$$

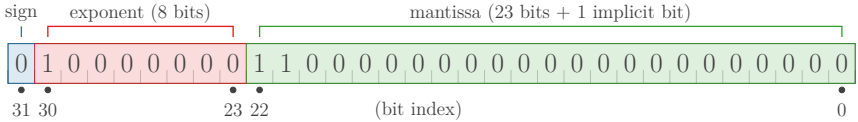
Let n_e be the number of bits in the exponent (here 8):

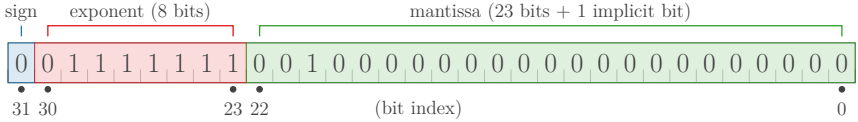
$$\mathcal{N} = s \times 2^{e-(2^{n_e-1}-1)} \times m = -1 \times 2^{67-127} \times 1.90625,$$

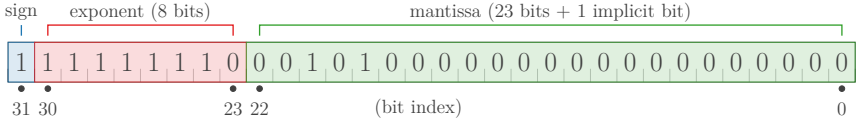
$$\mathcal{N} = -2^{-60} \times 1.90625 = -1.653408313 \times 10^{-18}.$$

32-bit Floating-point – Warmup Tasks

Give the values of the following IEEE 754 32 bits in base 10:







32-bit Floating-point – Zero & Infinity

Two representations for zero, the first one is +0 (all zeros):

0	0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

And the second one is −0 (all zeros except sign bit):

1	0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

32-bit Floating-point – Zero & Infinity

Two representations for zero, the first one is +0 (all zeros):

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

And the second one is −0 (all zeros except sign bit):

1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Infinity +∞ (unset sign bit, all ones exponent bits, all zeros mantissa):

0	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Minus infinity −∞ (set sign bit, all ones exponent bits, all zeros mantissa):

1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

By default, **dividing by Zero produces infinity**.

32-bit Floating-point – NaNs

- Bits sequence that represents something that is Not a Number (= `NaN`)
- Some examples of operations generating a `NaN`
  - Dividing zero by zero produces a `NaN`... `float nan = 0.f / 0.f;`
  - Square root and logarithm of a negative number
  - Inverse sine or inverse cosine of a number that is less than -1 or greater than 1
  - $\pm\infty / \pm\infty$, $\pm 0 \times \pm\infty$, $(+\infty) + (-\infty)$, ...
- The only value that is not equal to itself: `nan == nan` returns false!

$\texttt{NaN} \leq x$	$\texttt{NaN} < x$	$\texttt{NaN} \geq x$	$\texttt{NaN} > x$	$\texttt{NaN} = x$	$\texttt{NaN} \neq x$
`false`	`false`	`false`	`false`	`false`	`true`

Exponent bits are set to one and at least one bit of the mantissa is set to one:

0	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

`-NaN` is also defined when the sign bit is set to one.

32-bit Floating-point – NaNs

- Bits sequence that represents something that is Not a Number (`NaN`)
- Some examples of operations generating a `NaN`
  - Dividing zero by zero produces a `NaN`... `float nan = 0.f / 0.f;`
  - Square root and logarithm of a negative number
  - Inverse sine or inverse cosine of a number that is less than -1 or greater than 1
  - $\pm\infty / \pm\infty$, $\pm 0 \times \pm\infty$, $(+\infty) + (-\infty)$, ...
- The only value that is not equal to itself: `nan == nan` returns false!

$\texttt{NaN} \leq x$	$\texttt{NaN} < x$	$\texttt{NaN} \geq x$	$\texttt{NaN} > x$	$\texttt{NaN} = x$	$\texttt{NaN} \neq x$
false	false	false	false	false	true

Watch out, NaNs are super contagious!

Exponent bits are set to one and at least one bit of the mantissa is set to one:

0	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

`-NaN` is also defined when the sign bit is set to one.

32-bit Floating-point – Ranges

Smallest normalized number that can be represented (in absolute value):

0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

$$\mathcal{N}_{\text{smallest}} = 2^{1-127} \times 2^0 = 2^{-126} \approx 1.1754943 \times 10^{-38}$$

Biggest normalized number that can be represented (in absolute value):

0	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

$$\mathcal{N}_{\text{biggest}} = 2^{254-127} \times \sum_{i=0}^{23} 2^{-i} = 2^{127} \times (2 - 2^{-23}) \approx 3.4028236 \times 10^{38}$$

32-bit Floating-point – Some Usual Numbers

One, THE one:

0	0	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

$$\mathcal{N}_{\text{one}} = 2^{127-127} \times 2^0 = 2^0 \times 2^0 = 1 \times 1 = 1$$

Two:

0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

$$\mathcal{N}_{\text{two}} = 2^{128-127} \times 2^0 = 2^1 \times 2^0 = 2 \times 1 = 2$$

Three:

0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

$$\mathcal{N}_{\text{three}} = 2^{128-127} \times (2^0 + 2^{-1}_{b_{22}}) = 2^1 \times (1 + 0.5) = 2 \times 1.5 = 3$$

32-bit Floating-point – Representable Integers

$16\ 777\ 217 = 2^{24} + 1$:

0	1	0	0	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

$$\mathcal{N}_{\text{big1}} = 2^{151-127} \times (2^0 + 2^{-23}_{b_0}) = 2^{24} \times 1 = 16\ 777\ 216$$

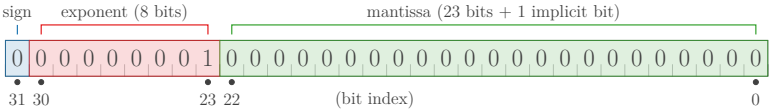
0	1	0	0	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

$$\mathcal{N}_{\text{big2}} = 2^{151-127} \times (2^0 + 2^{-23}_{b_0}) = 2^{24} \times 1.00000011920929 = 16\ 777\ 218$$

- 16 777 217 does not exist on 32-bit floats: Big integers are approximated!
- Happens when the integer is bigger than 2^{n_m+1} (with n_m the number of bits in the mantissa)
- For instance: 16 777 218 can be represented while 16 777 219 cannot

32-bit Floating-point – Subnormal Numbers (1)

- Smallest normalized number is $2^{-}126 = 1.1754943 \times 10^{-38}$



- But smaller numbers can be defined: **Subnormal numbers**
  - **All the exponent bits are set to 0** and the mantissa contains **at least one 1**
  - Exponent value is $-(2^{n_e-1} - 2) = -(2^7 - 2) =$ **-126** (and NOT -127, be careful), **-126 guarantees the continuity with normalized numbers**
  - **Implicit bit** b_{implicit} **is set to 0**

$$\mathcal{N}_{\text{subn}} = s \times 2^{\mathbf{-126}} \times m, \text{ where } m = \mathbf{0} + \sum_{i=1}^{23} b_{23-i} \times 2^{-i}.$$

32-bit Floating-point – Subnormal Numbers (2)

- **The biggest** subnormal number is:

0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

$$\mathcal{N}_{\text{subn biggest}} = s \times 2^{-126} \times (2^0 - 2^{-23}) \approx 1.1754942 \times 10^{-38}$$

- **The smallest** subnormal number is:

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

$$\mathcal{N}_{\text{subn smallest}} = s \times 2^{-126} \times 2^{-23} \approx 1.4012984 \times 10^{-45}$$

[1] Isaac Dooley and Laxmikant Kale. "Quantifying the Interference Caused by Subnormal Floating-Point Values". In: *Workshop on Operating System Interference in High Performance Applications*. Sept. 2006. URL: http://charm.cs.illinois.edu/newPapers/06-13/paper.pdf.

32-bit Floating-point – Subnormal Numbers (2)

- **The biggest** subnormal number is:

0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

$$\mathcal{N}_{\text{subn biggest}} = s \times 2^{-126} \times (2^0 - 2^{-23}) \approx 1.1754942 \times 10^{-38}$$

- **The smallest** subnormal number is:

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

$$\mathcal{N}_{\text{subn smallest}} = s \times 2^{-126} \times 2^{-23} \approx 1.4012984 \times 10^{-45}$$

- Be careful, these numbers **can executes slower on computers**
  - Because of **conditional management** and because **sometimes they are managed in software** ($\approx 5\times$ slower)![1] [2]

[1] Isaac Dooley and Laxmikant Kale. "Quantifying the Interference Caused by Subnormal Floating-Point Values". In: *Workshop on Operating System Interference in High Performance Applications*. Sept. 2006. URL: http://charm.cs.illinois.edu/newPapers/06-13/paper.pdf.

32-bit Floating-point – Overflow and Underflow

Overflow

- Finite result is too large to be represented accurately
- By default returns $\pm\infty$

Underflow

- Result is very small (outside the normal range)
- By default, returns a number less than or equal to the minimum positive normal number in magnitude (following the rounding rules)

Inexact

- The exact (i.e., unrounded) result is not representable exactly
- By default, returns the correctly rounded result

32-bit Floating-point – Underflow Problem

$2^{26} = 67\ 108\ 864$:

0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

$2^{1} = 1$:

0	0	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Problem: $67\ 108\ 864 + 1 = 67\ 108\ 864$ (if rounded to the nearest)!

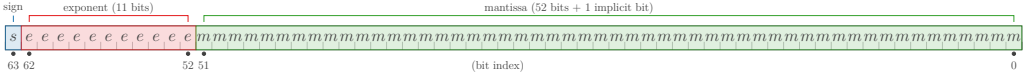
Next representable number is:

0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

$$\mathcal{N}_{\text{next}} = 2^{26} \times (1 + 2^{-23}) = 67\ 108\ 872$$

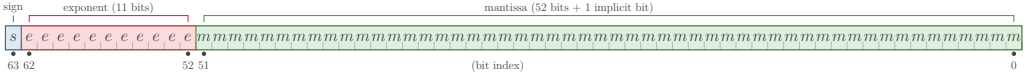
Other Floating-point Formats

- FP64 – Defined in IEEE 754-1985: $s = 64$ bits, $e = 11$ bits, $m = 52$ bits
  - Corresponding C data type: `float64_t` of `double` (for double precision)
  - Interval $[\pm 2^{-1022}, \pm 2^{1023} \times m]$, Bias $\mathcal{B} = 1023$
  - Available in most of the computers on the market
  - Generally used for accurate scientific simulations


sign
exponent (11 bits)
mantissa (52 bits + 1 implicit bit)
s
e e e e e e e e e e
m m m

Other Floating-point Formats

- FP64 – Defined in IEEE 754-1985: $s = 64$ bits, $e = 11$ bits, $m = 52$ bits
  - Corresponding C data type: `float64_t` of `double` (for double precision)
  - Interval $[\pm 2^{-1022}, \pm 2^{1023} \times m]$, Bias $\mathcal{B} = 1023$
  - Available in most of the computers on the market
  - Generally used for accurate scientific simulations


sign
exponent (11 bits)
mantissa (52 bits + 1 implicit bit)
s e e e e e e e e e e m m m

Conversions

TODO

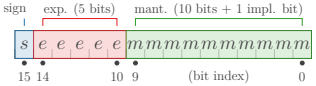
Table of Contents

3 AI Floating-point Formats for DNNs

- Integers
- Floating-point – IEEE 754
- AI Floating-point Formats for DNNs
- Fixed-point

Evolution through GPUs – 16 bits

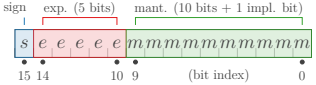
- Nvidia Pascal 2016 (GTX 10), Volta 2017 (V100) & Turing 2018 (RTX 20) ⇒ 2× **IEEE 754-2008 FP16** per cycle
  - Mainly used to compress 32-bit data in memory at the beginning
  - Then used in the tensor cores for IA computations (starting with Volta & Turing), in video games this was mainly used for image upscaling with CNNs



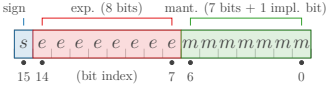
[1] Dhiraj D. Kalamkar et al. *A Study of BFLOAT16 for Deep Learning Training*. 2019. DOI: 10.48550/arXiv.1905.12322. arXiv: 1905.12322 [cs.LG]. URL: https://arxiv.org/pdf/1905.12322.

Evolution through GPUs – 16 bits

- Nvidia Pascal 2016 (GTX 10), Volta 2017 (V100) & Turing 2018 (RTX 20) ⇒ 2× **IEEE 754-2008 FP16** per cycle
  - Mainly used to compress 32-bit data in memory at the beginning
  - Then used in the tensor cores for IA computations (starting with Volta & Turing), in video games this was mainly used for image upscaling with CNNs



- Nvidia Ampere 2020 (RTX 30) ⇒ FP16 + new **Brain Float** (BF16)[1][4, 5, 6]
  - Designed for multi-precision DNN apps: Easier to adapt than FP16
  - Easy to convert to/from IEEE 754 FP32: Same interval $[\pm 2^{-126}; \pm 2^{127} \times m]$

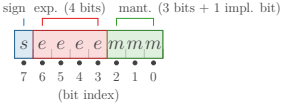


[1] Dhiraj D. Kalamkar et al. *A Study of BFLOAT16 for Deep Learning Training*. 2019. DOI: 10.48550/arXiv.1905.12322. arXiv: 1905.12322 [cs.LG]. URL: https://arxiv.org/pdf/1905.12322.

Evolution through GPUs – 8 bits E4M3

Nvidia Ada Lovelace 2022 (RTX 40) & Hopper (H100/H200)

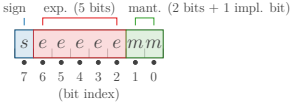
- FP16 + BF16 + **FP8 formats**[1]
- **Quasi-exclusively for DNNs** and especially for CNNs, RNNs, and Transformer-based models
- **FP8 E4M3** – $s = 8$ bits, $e = 4$ bits and $m = 3$ bits
  - $\mathcal{B} = 7$, Interval of normalized $\mathcal{N}$ is $[\pm 2^{-}6; \pm 2^{8} \times 1.75] = [\pm 0,015625; \pm 448]$
  - Support NaNs but NO $\pm\infty$
  - Interval of subnormal $\mathcal{N}$ is $[\pm 2^{-9}; \pm 2^{-6} \times 0.875]$



[1] Paulius Micikevicius et al. *FP8 Formats for Deep Learning*. 2022. DOI: 10.48550/arXiv.2209.05433. arXiv: 2209.05433 [cs.LG]. URL: https://arxiv.org/abs/2209.05433.

Evolution through GPUs – 8 bits E5M2

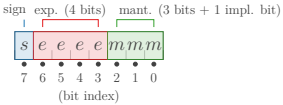
- **FP8 E5M2** – $s = 8$ bits, $e = 5$ bits and $m = 2$ bits
  - $\mathcal{B} = 15$, Interval of normalized $\mathcal{N}$ is $[\pm 2^{-14}; \pm 1.75 \times 2^{15}] = [\pm 2^{-14}; \pm 57\ 344]$
  - Support NaNs and $\pm\infty$
  - Interval of subnormal $\mathcal{N}$ is $[\pm 2^{-16}; \pm 2^{-14} \times 0.75]$
  - Can be seen as a **less precise IEEE 754-2008 FP16**
  - Nvidia documentation available[1]

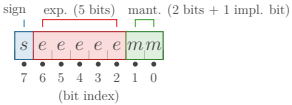


- During DNN training: FP8 E5M2 are more suitable for the backward pass while FP8 E4M3 are more suitable for the forward pass

[1] Nvidia. *Using FP8 with Transformer Engine*. Accessed on 2025-01-20. URL: https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/examples/fp8_primer.html.

Evolution through GPUs – FP8 E4M3 & E5M2



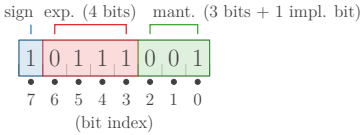


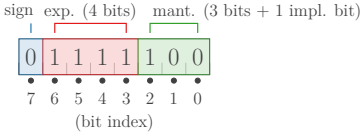
	FP8 E4M3	FP8 E5M2
Exponent bias ($\mathcal{B}$)	7	15
Infinities ($\pm\infty$)	N/A	$s.11111.00_2$
NaN	$s.1111.111_2$	$s.11111.\{01, 10, 11\}_2$
Zeros	$s.0000.000_2$	$s.00000.00_2$
Max normalized	$s.1111.110_2 = \pm 2^8 \times 1.75 = \pm 448$	$s.11110.11_2 = \pm 2^{15} \times 1.75 = \pm 57,344$
Min normalized	$s.0001.000_2 = \pm 2^{-6}$	$s.00001.00_2 = \pm 2^{-14}$
Max subnormal	$s.0000.111_2 = \pm 2^{-6} \times 0.875$	$s.00000.11_2 = \pm 2^{-14} \times 0.75$
Min subnormal	$s.0000.001_2 = \pm 2^{-6} \times 0.125 = \pm 2^{-9}$	$s.00000.01_2 = \pm 2^{-14} \times 0.25 = \pm 2^{-16}$

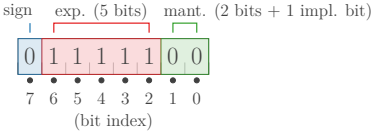
Relax, Take It Easy

3 AI Floating-point Formats for DNNs

Find the following number values in base 10:







Evolution through GPUs – Illustrative Example

Closest **estimation of 0.3952** depending on various floating-point formats:

FP64 | 0 | 0 1 1 1 1 1 1 1 1 0 1 | 1 0 0 1 0 1 0 0 1 0 1 0 1 1 1 1 0 1 0 0 1 1 1 1 0 0 0 0 1 1 0 1 1 0 0 0 0 1 0 0 0 1 0 0 1 1 0 1 0 0 0 0

$= 0.39519999999999999957367435854 39$

FP32 | 0 | 0 1 1 1 1 1 0 1 | 1 0 0 1 0 1 0 0 1 0 1 0 1 1 1 1 0 1 0 1 0 0 0 | $= 0.39520001411437988281 25$

FP16 | 0 | 0 1 1 0 1 | 1 0 0 1 0 1 0 0 1 1 | $= 2^{13-15} \times (2^{-0} + 2^{-1} + 2^{-4} + 2^{-6} + 2^{-9} + 2^{-10}) = 2^{-2} \times 1.5810546875 = 0.3952636719$

BF16 | 0 | 0 1 1 1 1 1 0 1 | 1 0 0 1 0 1 0 | $= 2^{125-127} \times (2^{-0} + 2^{-1} + 2^{-4} + 2^{-6}) = 2^{-2} \times 1.578125 = 0.39453125$

FP8 E4M3 | 0 | 0 1 0 1 | 1 0 1 | $= 2^{5-7} \times (2^{-0} + 2^{-1} + 2^{-3}) = 2^{-2} \times 1.625 = 0.40625$

FP8 E5M2 | 0 | 0 1 1 0 1 | 1 0 | $= 2^{13-15} \times (2^{-0} + 2^{-1}) = 2^{-2} \times 1.5 = 0.375$

Evolution through GPUs – 6 bits & 4 bits

Nvidia Blackwell 1 2024 (B100) & Blackwell 2 2025 (RTX 50/B200)

- FP16 + BF16 + FP8 + **FP6 & FP4**
- **FP6** – Lossy compression of FP8
  - Reduced memory occupancy + FP8 compute units are used

[1] Interesting blog post about FP4: https://www.zach.be/p/how-do-nvidia-blackwell-gpus-train

Evolution through GPUs – 6 bits & 4 bits

Nvidia Blackwell 1 2024 (B100) & Blackwell 2 2025 (RTX 50/B200)

- FP16 + BF16 + FP8 + **FP6 & FP4**
- **FP6** – Lossy compression of FP8
  - Reduced memory occupancy + FP8 compute units are used
- **FP4** – Only $2^4 = 16$ representable values!
  - Few documentation available at this time[1]
  - We should have: $e = 2$ bits and $m = 1$ bit and $\mathcal{B} = 1$
  - Array of data share **a high-precision scaling factor**

IEEE 754-compliant format

	$0..0_2$	$0..1_2$	$1..0_2$	$1..1_2$
$.00._2$	0.0	0.5	-0.0	-0.5
$.01._2$	1.0	1.5	-1.0	-1.5
$.10._2$	2.0	3.0	-2.0	-3.0
$.11._2$	$+\infty$	NaN	$-\infty$	NaN

Nvidia style format w/o $\pm\infty$ & NaNs

	$0..0_2$	$0..1_2$	$1..0_2$	$1..1_2$
$.00._2$	0.0	0.5	-0.0	-0.5
$.01._2$	1.0	1.5	-1.0	-1.5
$.10._2$	2.0	3.0	-2.0	-3.0
$.11._2$	4.0	6.0	-4.0	-6.0

[1] Interesting blog post about FP4: https://www.zach.be/p/how-do-nvidia-blackwell-gpus-train

Table of Contents

4 Fixed-point

- Integers
- Floating-point – IEEE 754
- AI Floating-point Formats for DNNs
- Fixed-point

Introduction to Fixed-point Format

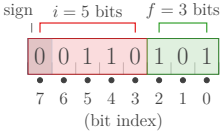
- A format where the **position of the decimal point** in the number **is fixed**
- In this format there are $\boldsymbol{i}$ **bits for integer part**, $\boldsymbol{f}$ **bits for the fractional part** and the capacity is $s = i + f$ bits

$$\mathcal{N} = i + f, \text{ where } i = -b_{s-1} \times 2^{i-1} + \sum_{ii=0}^{i-2} b_{ii+f} \times 2^{ii}, \text{ and } f = \sum_{ff=1}^{f} b_{ff-f} \times 2^{-ff}.$$

Introduction to Fixed-point Format

- A format where the **position of the decimal point** in the number **is fixed**
- In this format there are $\boldsymbol{i}$ **bits for integer part**, $\boldsymbol{f}$ **bits for the fractional part** and the capacity is $s = i + f$ bits

$$\mathcal{N} = i + f, \text{ where } i = -b_{s-1} \times 2^{i-1} + \sum_{ii=0}^{i-2} b_{ii+f} \times 2^{ii}, \text{ and } f = \sum_{ff=1}^{f} b_{ff-f} \times 2^{-ff}.$$



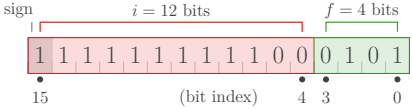
$$i = -b_7 \times 2^4 + b_6 \times 2^3 + b_5 \times 2^2 + b_4 \times 2^1 + b_3 \times 2^0 = 2^2 + 2^1 = 6$$

$$f = b_2 \times 2^{-1} + b_1 \times 2^{-2} + b_0 \times 2^{-3} = 2^{-1} + 2^{-3} = 0.625$$

$$\mathcal{N} = i + f = 6.625$$

Signed Numbers with Two's Complement

- **Two's complement** for the integer part i
- Example of **a negative number on 16 bits** ($s = 16$)



$$i = -b_{15} \times 2^{11} + b_{14} \times 2^{10} + b_{13} \times 2^{9} + b_{12} \times 2^{8} + b_{11} \times 2^{7} + b_{10} \times 2^{6} + b_{9} \times 2^{5} + b_{8} \times 2^{4} + b_{7} \times 2^{3} + b_{6} \times 2^{2}$$

$$i = -2048 + (1024 + 512 + 256 + 128 + 64 + 32 + 16 + 8 + 4) = -4$$

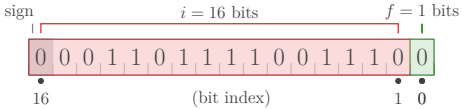
$$f = b_2 \times 2^{-2} + b_0 \times 2^{-4} = 0.3125$$

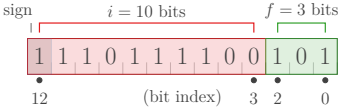
$$\mathcal{N} = i + f = -4 + 0.3125 = -3.6875$$

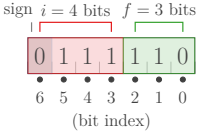
Warmup Tasks

4 Fixed-point

Find out these numbers in base 10:

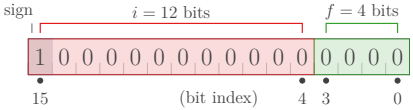






Interval of Representable Values

- Smallest value

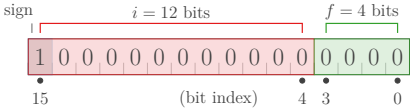


— $\mathcal{N}_{\text{smallest}} = -2^{i-1} = -2^{11} = -2048$

Interval of Representable Values

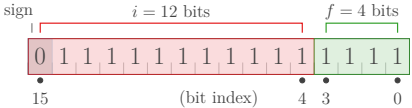
4 Fixed-point

- Smallest value



  — $\mathcal{N}_{\text{smallest}} = -2^{i-1} = -2^{11} = -2048$

- Biggest value



  — $\mathcal{N}_{\text{biggest}} = 2^{i-1} - 1 + \sum_{ff=1}^{f} b_{ff-f} \times 2^{-ff} =$
  $(2^{11} - 1) + (2^{-1} + 2^{-2} + 2^{-3} + 2^{-4} = 2047 + 0.9375 = 2047.9375$

Distance between Representable Numbers

4 Fixed-point

0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	$= 0.9375$
0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	$= 1$ and $1 - 0.9375 = 0.0625$
0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	$= 1.0625$ and $1.0625 - 1 = 0.0625$
0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	$= 1.125$ and $1.125 - 1.0625 = 0.0625$
0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	$= 1.1875$ and $1.1875 - 1.125 = 0.0625$

Contrary to the floating-point representations, the **distance between two consecutive numbers is always constant**: $\mathcal{D} = 2^{-f}$ (here $2^{-4} = 0.0625$)

Distance between Representable Numbers

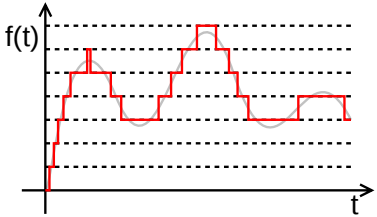
4 Fixed-point

0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	$= 0.9375$
0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	$= 1$ and $1 - 0.9375 = 0.0625$
0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	$= 1.0625$ and $1.0625 - 1 = 0.0625$
0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	$= 1.125$ and $1.125 - 1.0625 = 0.0625$
0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	$= 1.1875$ and $1.1875 - 1.125 = 0.0625$

Contrary to the floating-point representations, the **distance between two consecutive numbers is always constant**: $\mathcal{D} = 2^{-f}$ (here $2^{-4} = 0.0625$)

- This is true on the chosen **uniform fixed-point representation** but other representations can be different
- Direct consequence is that the **numbers interval is much smaller than with floating-point** representations

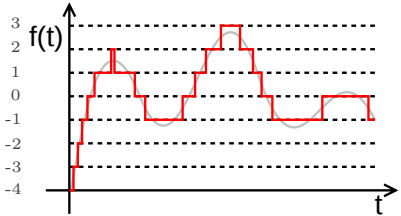
Quantization – Concept



In gray a continuous signal and in red a quantized version on 3 bits (from Wikipedia).

"Quantization, in mathematics and digital signal processing, is the process of mapping input values from a large set (often a continuous set) to output values in a (countable) smaller set, often with a finite number of elements."

Quantization – Concept



In gray a continuous signal and in red a quantized version on 3 bits (from Wikipedia).

"Quantization, in mathematics and digital signal processing, is the process of mapping input values from a large set (often a continuous set) to output values in a (countable) smaller set, often with a finite number of elements."

- Generally we want to approximate a continuous signal with a reduced number of values
- Rounding and truncation are typical examples of quantization process

Quantizer – From Floating-point to Fixed-point

How to **convert** a high precision **floating-point number into** a lower precision **fixed-point number**?

- We consider that the floating-point number $\mathcal{N}$ is close to a continuous value

$$\mathcal{Q}_{s,f} = \min(\max(\lfloor 2^f \times \mathcal{N} \rceil, -2^{s-1}), 2^{s-1} - 1)$$

Quantizer – From Floating-point to Fixed-point

How to **convert** a high precision **floating-point number into** a lower precision **fixed-point number**?

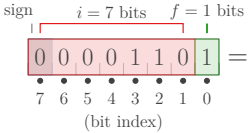
- We consider that the floating-point number $\mathcal{N}$ is close to a continuous value

$$\mathcal{Q}_{s,f} = \min(\max(\lfloor 2^f \times \mathcal{N} \rceil, -2^{s-1}), 2^{s-1} - 1)$$

- Let's take an example: $\mathcal{N} = 6.643$ and let's consider uniform fixed-point format where $i = 7$ and $f = 1$

$$\mathcal{Q}_{8,1} = \min(\max(\lfloor 2^f \times 6.643 \rceil, -2^{s-1}), 2^{s-1} - 1)$$

$$\mathcal{Q}_{8,1} = \min(\max(\lfloor 13.286 \rceil, -128), +127) = 13$$



$$= 2^2 + 2^1 + 2^{-1} = 6.5$$

Quantization – Re-scaling (1)

Problem: The range of the representable values is mainly limited by the integer part $[-2^{i-1}; 2^{i-1}-1]$.

- For instance, if $s = 8$ and $i = 7$, the range is $[-64; 63.5]$
  - If the number we want to quantize is bigger than 63.5, then it will be saturated
  - If the number we want to quantize is smaller than -64, then it will be saturated
- Let's suppose that we have continuous numbers uniformly distributed in the $[-128; 127]$ range
  - If we consider the previous quantizer ($\mathcal{Q}_{s,f} = \min(\max(\lfloor 2^f \times \mathcal{N} \rceil, -2^{s-1}), 2^{s-1}-1)$, the problem is that half of the values will be saturated (= info loss)
  - Introducing a re-scaling factor r

$$\mathcal{Q}^r_{s,f} = \min(\max(\lfloor \frac{2^f \times \mathcal{N}}{r} \rceil, -2^{s-1}), 2^{s-1}-1).$$

Quantization – Re-scaling (2)

$$\mathcal{Q}^r_{s,f} = \min(\max(\lfloor \frac{2^f \times \mathcal{N}}{r} \rceil, -2^{s-1}), 2^{s-1} - 1).$$

- Let's suppose that we have continuous numbers uniformly distributed in the $[-128; 127]$ range
  - If we consider we have the value -127, $s = 8$ bits & $f = 1$ bit and a re-scaling factor $r = 2$

$$\mathcal{Q}^2_{8,1} = \min(\max(\lfloor \frac{2^1 \times -127}{2} \rceil, -128), 127)$$

$$\mathcal{Q}^2_{8,1} = \min(\max(-127, -128), 127) = -127$$

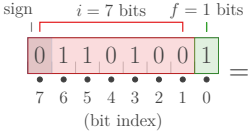
- In fact $\mathcal{Q}^2_{8,1} = \mathcal{Q}^1_{8,0}$, in other words the standard 8-bit integer format `:-D`!
  - We just found that the best way to represent uniformly distributed valued between -128 and -127 is to use 8-bit integers

Quantization – Re-scaling (3)

- Let's suppose that we have continuous numbers uniformly distributed in the $[-96; 95]$ range, now it becomes more interesting
- Still if we consider a $\mathcal{Q}_{8,1}$ format, we can apply $r = 1.5$ ($\mathcal{Q}_{8,1}^{1.5}$) to re-scale the values as follow $[-64 \times 1.5; 63 \times 1.5] = [-96; 94.5]$
- If $\mathcal{N} = 79$

$$\mathcal{Q}_{s,f}^{r} = \min(\max(\lfloor \frac{2^f \times \mathcal{N}}{r} \rceil, -2^{s-1}), 2^{s-1} - 1),$$

$$\mathcal{Q}_{8,1}^{1.5} = \min(\max(\lfloor \frac{2^1 \times 79}{1.5} \rceil, -128), 127) = \min(\max(\lfloor \frac{158}{1.5} \rceil, -128), 127) = \lfloor 105.333333 \rceil = 105$$



$$= (2^5 + 2^4 + 2^2 + 2^{-1}) \times 1.5 = 52.5 \times 1.5 = 78.75$$

- With $\mathcal{Q}_{8,1}^{1.5}$, the distance between two consecutive numbers is 0.75 ($\mathcal{D} = 2^{-f} \times r$)

Logic & Arithmetic Operations

Good news, some logic and arithmetic operations work out of the box!

- Integer comparisons are working
- Integer addition and subtraction are working
- Multiplication & division are generally NOT working
  - Multiplying random number together requires additional instructions or a dedicated HW
  - Division and multiplication by power of two are working

Q&A

Thank you for listening!
Do you have any questions?

Bibliography

5 References

[1] Isaac Dooley and Laxmikant Kale. "Quantifying the Interference Caused by Subnormal Floating-Point Values". In: *Workshop on Operating System Interference in High Performance Applications*. Sept. 2006. URL: http://charm.cs.illinois.edu/newPapers/06-13/paper.pdf.

[2] Wikipedia. *Subnormal Number*. Accessed on 2025-01-20, see "Performance issues" section. URL: https://en.wikipedia.org/wiki/Subnormal_number.

[3] Dhiraj D. Kalamkar et al. *A Study of BFLOAT16 for Deep Learning Training*. 2019. DOI: 10.48550/arXiv.1905.12322. arXiv: 1905.12322 [cs.LG]. URL: https://arxiv.org/pdf/1905.12322.

[4] Arne Verheyde. *BFloat16 Deep Dive: ARM Brings BF16 Deep Learning Data Format to ARMv8-A*. Accessed on 2025-01-20. 2019. URL: https://www.tomshardware.com/news/bfloat16-deep-dive-arm-bf16-support-armv8-a,40305.html.

[5] Nvidia. *Nvidia Ampere GA102 GPU Architecture*. Accessed on 2025-01-20. URL: https://images.nvidia.com/aem-dam/en-zz/Solutions/geforce/ampere/pdf/NVIDIA-ampere-GA102-GPU-Architecture-Whitepaper-V1.pdf.

[6] Nigel Stephens. *BFloat16 Processing for Neural Networks on Armv8-A*. Accessed on 2025-01-20. 2019. URL: https://community.arm.com/arm-community-blogs/b/ai-and-ml-blog/posts/bfloat16-processing-for-neural-networks-on-armv8_2d00_a.

Bibliography

5 References

[7] Paulius Micikevicius et al. *FP8 Formats for Deep Learning*. 2022. DOI: 10.48550/arXiv.2209.05433. arXiv: 2209.05433 [cs.LG]. URL: https://arxiv.org/abs/2209.05433.

[8] Nvidia. *Using FP8 with Transformer Engine*. Accessed on 2025-01-20. URL: https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/examples/fp8_primer.html.